

Testa – AI QA Agent: Technical Architecture & Learning Guide

A detailed reference for understanding how Testa was built, why each technology was chosen, and how to build similar custom self-hosted AI agents from scratch.

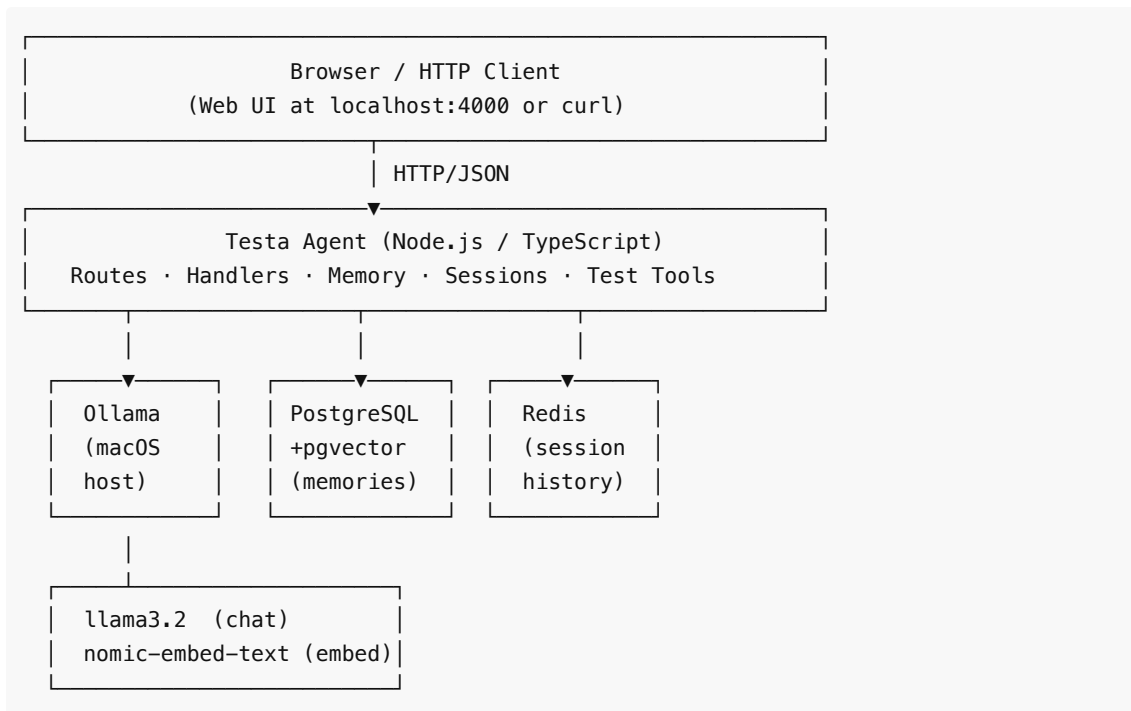
Table of Contents

1. [System Overview](#)
 2. [How AI Works — The Core Concepts](#)
 - 2.1 Large Language Models (LLMs)
 - 2.2 Embeddings and Vector Space
 - 2.3 Cosine Similarity and Distance
 - 2.4 Retrieval-Augmented Generation (RAG)
 - 2.5 Attention, Weights, and Temperature
 - 2.6 Prompt Engineering
 3. [Architecture Diagram](#)
 4. [Component Deep-Dives](#)
 - 4.1 Ollama — Local LLM Runtime
 - 4.2 llama3.2 — The Chat Model
 - 4.3 nomic-embed-text — The Embedding Model
 - 4.4 PostgreSQL + pgvector — Long-Term Memory
 - 4.5 HNSW Index — Fast Approximate Search
 - 4.6 Redis — Short-Term Conversational Memory
 - 4.7 Node.js + TypeScript — The Agent Server
 - 4.8 Docker + Docker Compose — Infrastructure
 5. [Agent Subsystems](#)
 - 5.1 Memory Layer (memory.ts)
 - 5.2 Session Layer (session.ts)
 - 5.3 LLM Client (ollama.ts)
 - 5.4 Knowledge Ingestion (ingest.ts)
 - 5.5 Test Engineering Tools (testtools.ts)
 - 5.6 Automation Browser (automation.ts)
 6. [Prompt Engineering in Testa](#)
 7. [Weighting, Confidence, and Tuning](#)
 8. [Technology Alternatives — Pros and Cons](#)
 9. [Building a New Custom Agent — Blueprint](#)
 10. [Glossary](#)
-

1. System Overview

Testa is a **domain-specific AI agent** that answers questions about a specific QA automation project, generates Playwright test code, reviews existing tests, and diagnoses test failures. It runs entirely on your local machine — no API keys, no cloud calls, no data leaving your network.

At a high level, there are five layers:



Data flows for a typical chat request:

1. User sends a question via the web UI.
2. The agent generates an embedding of the question (768 numbers representing its meaning).
3. PostgreSQL finds the 6 most semantically similar memories using vector cosine distance.
4. The agent loads the last 10 conversation turns from Redis.
5. It assembles a prompt: system persona + injected memories + conversation history + user question.
6. Ollama runs the LLM locally and streams back a response.
7. Both the question and answer are stored in Redis for future context.
8. The response is returned to the browser with a list of which memories were used.

2. How AI Works — The Core Concepts

This section explains the underlying AI mechanics that power Testa. Understanding these will let you tune, extend, and debug any AI agent.

2.1 Large Language Models (LLMs)

An LLM is a neural network trained to predict the next word (token) in a sequence. "Large" refers to the number of **parameters** (weights) — llama3.2 has ~3 billion parameters; GPT-4 has an estimated 1.7 trillion.

How training works (simplified):

1. The model is shown billions of text examples from the internet, books, and code.
2. For each position in a sentence, it predicts the next token.
3. The prediction is compared to the actual next token. The difference is the **loss**.
4. Through **backpropagation**, the loss signal is used to slightly adjust millions of weights so the model would predict better next time.
5. This repeats billions of times until the loss is minimised.

The result is a network where the weights have encoded an enormous amount of statistical knowledge about language, facts, and reasoning patterns.

At inference time (when you ask a question):

- The model reads your prompt as a sequence of tokens.
- It runs the tokens through many layers of matrix multiplication using its weights.
- It produces a probability distribution over the entire vocabulary for the next token.
- It samples from that distribution and emits a token.
- It feeds that token back in and repeats until it produces a stop token.

Key parameters that control generation:

Parameter	What it does	Default in Testa
temperature	How random the sampling is. 0 = always pick the most likely token (deterministic). 1+ = more creative/random.	Not set (Ollama default ~0.8)
top_p	Only sample from tokens whose cumulative probability reaches this value. Lower = more focused.	Not set
top_k	Only consider the top K most likely tokens at each step.	Not set
num_ctx	Context window size — how many tokens the model can "see" at once. llama3.2 supports up to 128K.	Not set (Ollama default)
seed	Fixed seed for reproducible outputs.	Not set

These can be set per-request in the `/api/chat` payload sent to Ollama.

2.2 Embeddings and Vector Space

An **embedding** is a fixed-size list of floating-point numbers (a vector) that represents the *meaning* of a piece of text, not its words.

The `nomic-embed-text` model converts any text into a 768-dimensional vector. Texts with similar meaning end up close together in this 768-dimensional space, regardless of the specific words used.

Example:

```
"The booking was confirmed" → [0.12, -0.34, 0.87, ..., 0.05] (768 numbers)
"The reservation was approved" → [0.11, -0.33, 0.89, ..., 0.06] (very similar)
"The cat sat on the mat" → [-0.78, 0.21, -0.44, ..., 0.91] (very different)
```

Why 768 dimensions? This is a design choice by the model authors. More dimensions = more expressive power but slower to compute. `nomic-embed-text` uses 768 because it balances quality and speed for domain-specific retrieval tasks. Other models use 1536 (OpenAI ada-002), 3072 (OpenAI text-3-large), or 384 (all-MiniLM-L6-v2).

2.3 Cosine Similarity and Distance

Given two vectors (two pieces of text), how do we measure how similar they are?

Cosine similarity measures the angle between two vectors:

$$\text{similarity} = (A \cdot B) / (|A| \times |B|)$$

Where $A \cdot B$ is the dot product and $|A|$ is the magnitude (length) of the vector.

- Cosine similarity of **1.0** = identical direction = same meaning
- Cosine similarity of **0.0** = perpendicular = unrelated
- Cosine similarity of **-1.0** = opposite direction = opposite meaning

Cosine distance = `1 - cosine_similarity` . This is what pgvector uses with the `<=>` operator.

In `memory.ts` :

```
// Lower distance = more similar
ORDER BY embedding <=> $1::vector
```

The `similarity` score returned to the UI is:

```
similarity = 1 - cosine_distance
```

A similarity of `0.85` means the memory is 85% aligned in meaning with the question.

Why cosine and not Euclidean distance? Euclidean distance is affected by the magnitude (length) of vectors, not just direction. Two texts with the same meaning but one being 10x longer would appear far apart in Euclidean space. Cosine only cares about direction (meaning), making it more robust for text.

2.4 Retrieval-Augmented Generation (RAG)

RAG is the core pattern that makes Testa domain-aware. Without RAG, the LLM only knows what it learned during training (which doesn't include your specific project). With RAG:

```
Question → Embed → Search memories → Retrieve top-K relevant chunks
                                   ↓
                               Inject into prompt as "CURRENT KNOWLEDGE"
                                   ↓
                               LLM answers using that context
```

This is powerful because:

- The LLM's knowledge is extended with project-specific facts at query time.
- You can update knowledge without retraining the model.
- You can see exactly which memories influenced an answer.
- The model can cite its sources.

Testa's RAG pipeline (in `index.ts` → `handleChat`):

```
// 1. Embed the user's question
// 2. Search for 6 most relevant memories
const memories = await recallRelevant(message, 6);

// 3. Format memories as a knowledge block
const knowledge = memories.map(m => `${m.memoryType} ${m.content}`).join("\n");

// 4. Inject into the chat call
const answer = await chat(messages, knowledge);
```

In `ollama.ts` , the knowledge is injected into the system message:

```
CURRENT PRODUCT KNOWLEDGE (retrieved from memory, use this to inform your answer):  
[feature] The booking creation form requires a business unit and job to be selected...  
[architecture] The BookingsPage class has methods: startCreateBooking(),  
selectBusinessUnit()...
```

2.5 Attention, Weights, and Temperature

Attention (the "Transformer" in LLM): Every LLM built since 2017 is based on the Transformer architecture. The key innovation is **self-attention**: for each token in the input, the model learns which other tokens are most relevant to understanding it.

The attention mechanism computes three matrices from the input:

- **Query (Q)** — "what am I looking for?"
- **Key (K)** — "what do I have?"
- **Value (V)** — "what should I return?"

Attention score = $\text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$

This lets the model understand that in "The booking was cancelled because it exceeded the RTW limit", the word "it" refers to "booking" not "limit", without any hard-coded rules.

Weights: Every operation in the neural network is controlled by learned weights — matrices of floating-point numbers. A 3B parameter model has about 3 billion such numbers. When you load Llama3.2 into Ollama, it loads this entire weight matrix from disk. The weights are fixed at inference time — they don't change while you're chatting.

Temperature: Temperature controls the randomness of the probability sampling:

Low temperature (0.1–0.3):

Token probs: "the"=0.60, "a"=0.25, "this"=0.10, "an"=0.05

After temp: "the"=0.95, "a"=0.04, "this"=0.01

→ Almost always picks "the" → Predictable, factual, repetitive

High temperature (1.2–1.5):

After temp: "the"=0.45, "a"=0.30, "this"=0.15, "an"=0.10

→ More varied picks → More creative, less predictable, higher error rate

For a QA agent generating test code, lower temperature (~0.3–0.5) produces more consistent, correct output. For brainstorming test ideas, higher temperature gives more variety.

2.6 Prompt Engineering

The prompt is the primary control surface for LLM behaviour. Testa uses several proven techniques:

System prompt — defines persona and hard rules:

You are Testa, an AI-powered QA engineer...

NEVER invent product behaviour you haven't been told about.

When uncertain, say "I don't know for certain, but based on what I know..."

Structured output prompting — forces a specific format:

Return ONLY raw TypeScript code. No markdown code fences. No explanations.

Few-shot examples — shows the model exactly what good output looks like by including 1–2 real examples in the prompt. Testa includes a complete real test file as an example when generating tests. This dramatically improves output quality.

Role + context injection — gives the model the information it needs:

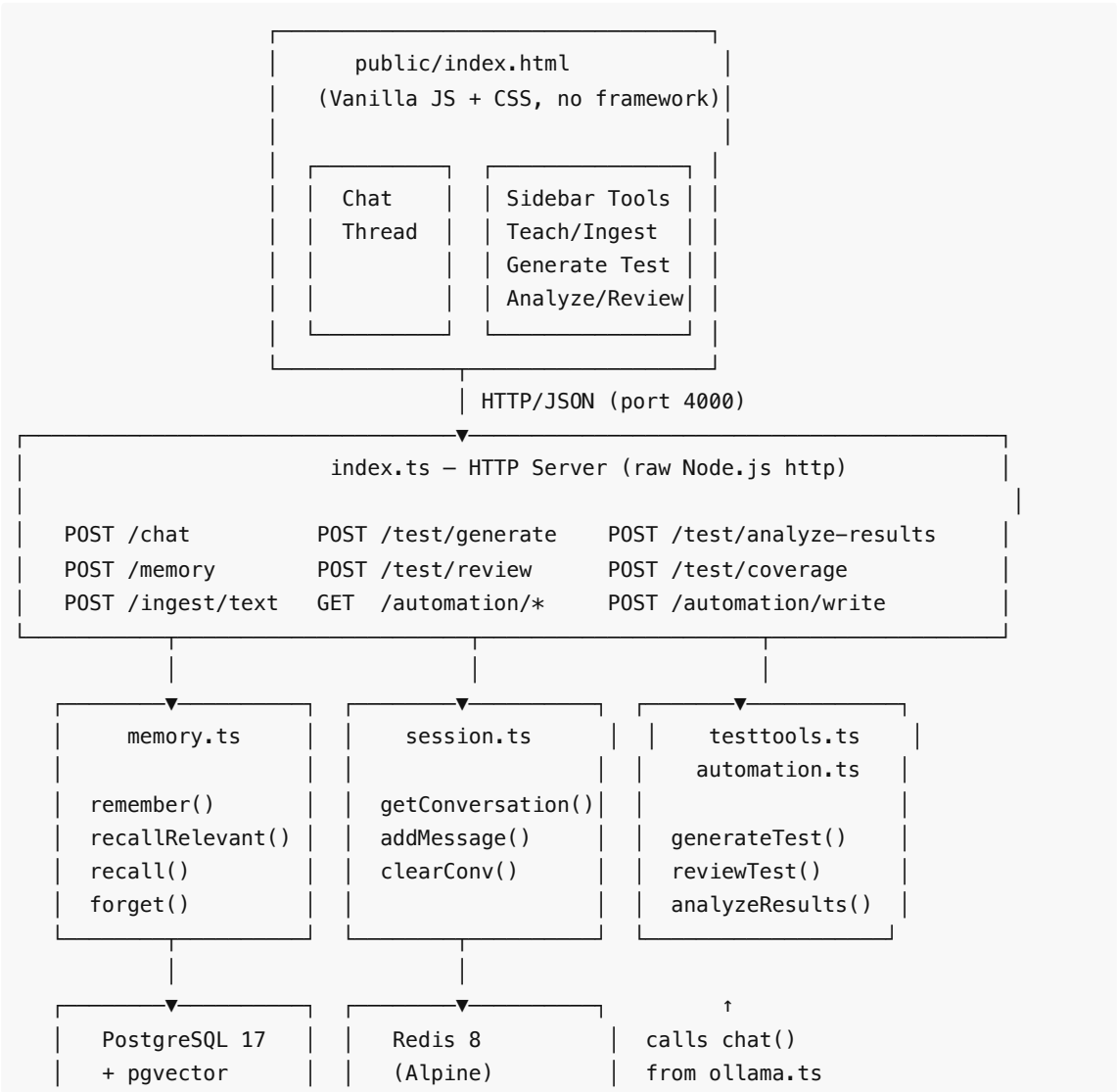
```
=== AVAILABLE PAGE OBJECTS ===
BookingsPage: startCreateBooking(), selectBusinessUnit(name), ...

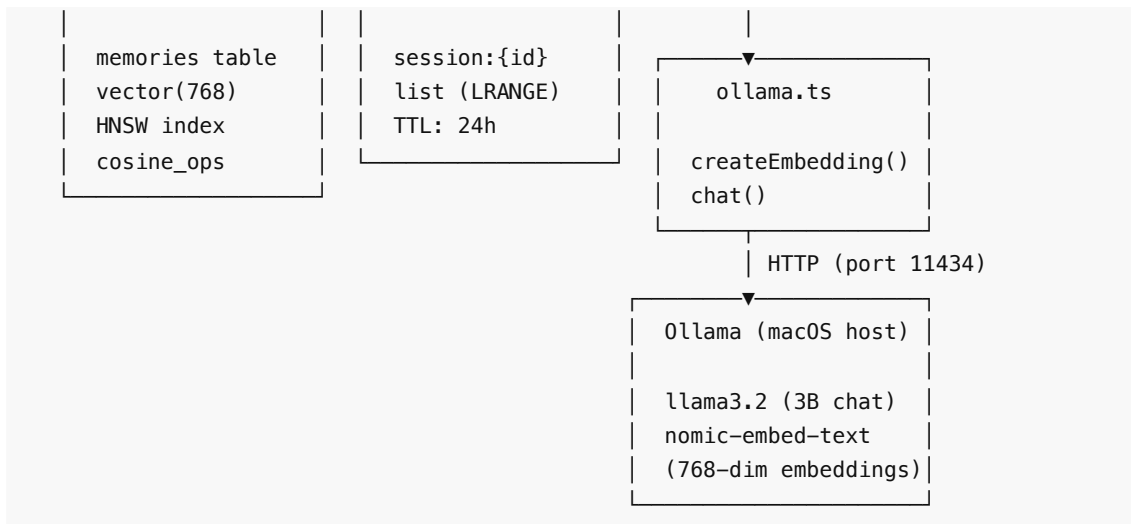
=== KNOWN PRODUCT FACTS ===
[feature] Bookings require a BU and Job to be pre-selected...
```

Chain of thought — asking the model to reason before answering (not currently used in Testa but a common technique):

```
Think through the problem step by step before giving your answer.
```

3. Architecture Diagram





4. Component Deep-Dives

4.1 Ollama — Local LLM Runtime

What it is: Ollama is a tool for running open-source LLMs locally on your machine. It provides:

- A REST API (`http://localhost:11434`) that mimics OpenAI's API shape
- GPU acceleration (Metal on Apple Silicon, CUDA on NVIDIA)
- Model management (`ollama pull` , `ollama list` , `ollama rm`)
- Automatic quantisation — models are stored in GGUF format with INT4/INT8 weights to reduce memory usage

Why it was chosen:

- Zero cost — completely free with no API keys
- Privacy — all inference happens locally, data never leaves the machine
- Apple Silicon optimisation — uses Metal GPU for fast inference on M-series Macs
- Simple REST API — trivial to call from any language
- Model flexibility — swap models by changing one environment variable

How the agent calls it (`src/ollama.ts`):

```

// Embed a piece of text
const response = await fetch(`${OLLAMA_BASE_URL}/api/embed`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ model: EMBED_MODEL, input: text })
});

// Chat completion
const response = await fetch(`${OLLAMA_BASE_URL}/api/chat`, {
  method: "POST",
  body: JSON.stringify({
    model: chatModel,
    messages: messagesWithSystem,
    stream: false // wait for complete response
  })
});
  
```

```
}  
});
```

Ollama in Docker: The agent runs in Docker but Ollama runs on the Mac host. The `extra_hosts:` `host.docker.internal:host-gateway` directive in docker-compose makes the Mac's `localhost` reachable from inside the container at the hostname `host.docker.internal`. The `OLLAMA_BASE_URL` is set to `http://host.docker.internal:11434`.

4.2 llama3.2 — The Chat Model

What it is: Meta's Llama 3.2 3B parameter model, quantised to INT4 (Q4_K_M) by Ollama. The unquantised model would need ~6GB RAM; with INT4 quantisation it fits in ~2GB.

Key properties:

- 3B parameters — small enough to run in real-time on an M-series Mac without GPU saturation
- 128K token context window — can process very long prompts (the full test file + memories + history)
- Trained on code, English text, and structured data — good at TypeScript generation
- Instruction-tuned — trained to follow instructions, not just predict text

Quantisation explained: LLMs store weights as 32-bit or 16-bit floats. Quantisation converts these to 4-bit or 8-bit integers:

```
Weight: 0.183471... (float32, 4 bytes)  
→ Quantised: 3 (int4, 0.5 bytes)
```

This 8x compression reduces RAM usage and speeds up inference at the cost of slight accuracy loss. For most tasks the quality difference is unnoticeable.

Model file: Stored at `~/ollama/models/` on the Mac host as a single `.gguf` file.

4.3 nomic-embed-text — The Embedding Model

What it is: An open-source embedding model designed specifically for retrieval tasks. Produces 768-dimensional vectors.

Why 768 dimensions: nomic-embed-text was trained to balance three things:

- **Quality** — captures semantic meaning accurately
- **Speed** — 768 dimensions is fast to compute and index
- **Storage** — each embedding takes $768 \times 4 \text{ bytes} = 3\text{KB}$ of storage

How embeddings are stored:

```
-- In PostgreSQL, each memory has a vector column  
embedding vector(768)  
  
-- When we store a memory:  
INSERT INTO memories (content, embedding)  
VALUES ($1, $2::vector)  
  
-- When we search, we compute the cosine distance
```

```
-- between the query embedding and all stored embeddings
ORDER BY embedding <=> $queryEmbedding::vector
LIMIT 6
```

Context length: nomic-embed-text can embed up to 8192 tokens. Text longer than this is truncated. The ingestion pipeline's `chunkText()` function pre-splits text into ~400 character chunks to stay well within this limit and get more precise embeddings.

4.4 PostgreSQL + pgvector — Long-Term Memory

PostgreSQL is the relational database. **pgvector** is an extension that adds a `vector` column type and vector search operators to PostgreSQL.

Why PostgreSQL + pgvector instead of a dedicated vector database:

- PostgreSQL is already a world-class database with transactions, ACID guarantees, full-text search, and JSONB
- No additional service to learn, operate, or maintain
- Memories can be combined with relational queries (e.g., `WHERE memory_type = 'bug'`)
- pgvector is production-ready and used at scale (Supabase, GitHub Copilot)

The memories table schema:

```
CREATE TABLE memories (
  id          UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  content     TEXT NOT NULL,
  memory_type VARCHAR(50) NOT NULL DEFAULT 'fact',
  source      VARCHAR(100),
  confidence  NUMERIC(4,3) DEFAULT 1.000, -- 0.000 to 1.000
  metadata    JSONB DEFAULT '{}',
  embedding   vector(768),                -- the semantic fingerprint
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW()
);
```

Indexes:

```
-- For filtering by type (fast exact match)
CREATE INDEX idx_memories_type ON memories(memory_type);

-- For recent-first listing
CREATE INDEX idx_memories_created ON memories(created_at DESC);

-- For fast approximate nearest-neighbour search
CREATE INDEX idx_memories_embedding ON memories
  USING hnsu (embedding vector_cosine_ops);
```

4.5 HNSW Index — Fast Approximate Search

Without an index, finding the 6 most similar embeddings out of 10,000 memories would require computing the cosine distance to all 10,000 — a sequential scan. At scale this becomes too slow.

HNSW (Hierarchical Navigable Small World) is an approximate nearest-neighbour algorithm that builds a multi-layer graph of vectors:

```
Layer 2 (sparse):  A ————— G ————— M
                  |                               |
Layer 1 (medium):  A — C — E — G — I — K — M
                  |   |   |   |   |   |   |
Layer 0 (dense):   A-B-C-D-E-F-G-H-I-J-K-L-M-...
```

How search works:

1. Start at a random node in the top (sparse) layer
2. Greedily move toward the query vector by following the "closest neighbour" edges
3. Drop down to the next layer and repeat
4. At layer 0, return the top-K closest nodes found

This finds approximate (not exact) results in $O(\log n)$ time instead of $O(n)$, trading a tiny amount of accuracy for massive speed gains.

Configuration options for the HNSW index:

```
CREATE INDEX ... USING hnsw (embedding vector_cosine_ops)
WITH (
  m = 16,           -- max connections per node (higher = better quality, more
memory)
  ef_construction = 64 -- search width during index build (higher = better quality,
slower build)
);
```

For `ef_search` (search quality at query time), set:

```
SET hnsw.ef_search = 100; -- default 40; higher = better recall, slower
```

Testa uses default settings. If memory retrieval accuracy needs improvement, increase `ef_search`.

4.6 Redis — Short-Term Conversational Memory

Why Redis for sessions and not PostgreSQL:

- Redis is an in-memory data structure server — writes and reads are microseconds vs. milliseconds for disk-based PostgreSQL
- The `RPush` + `LRange` commands are purpose-built for maintaining ordered lists (conversation history)
- TTL (`EXPIRE`) is a first-class feature — sessions automatically expire after 24 hours with zero cleanup logic
- A conversation has no analytical value; it doesn't need ACID, indexes, or relationships

Session data structure:

```
Redis key:  session:a1b2c3d4-...
Type:      List
```

```
Structure: [msg1, msg2, msg3, ..., msgN]
TTL:      86400 seconds (24 hours)
```

Each message is stored as a JSON string: `{"role":"user","content":"How do I..."}` .

Retrieval:

```
// LRange session:id -10 -1 = last 10 messages (tail of list)
const rawMessages = await redis.lRange(`session:${sessionId}`, -limit, -1);
```

Why a rolling window of 10? LLMs have a context window limit. Including the full conversation history could overflow the context. 10 turns (~2000 tokens) provides enough context without consuming the token budget that should go to the answer.

4.7 Node.js + TypeScript — The Agent Server

Why Node.js:

- Native HTTP support — no framework needed for a simple API server
- TypeScript first-class — excellent tooling for typed interfaces and `async/await`
- Single-threaded event loop — perfect for an I/O-bound agent server that spends most of its time waiting on Ollama, PostgreSQL, and Redis
- Native ESM module support — `import / export` without transpilation complexity
- Tiny footprint — the agent's compiled output is ~300KB vs. a Java or Go service

Why no Express/Fastify: The routing is simple and static. A raw `http.createServer` with URL-pattern matching is ~20 lines and has no dependencies. Using a framework would add complexity and dependencies for no benefit.

TypeScript configuration:

```
{
  "target": "ES2022",           // Use modern JS features (async/await, ??, ?.)
  "module": "NodeNext",       // Native ESM (import/export)
  "strict": true,              // Catch null/undefined errors at compile time
  "outDir": "dist"             // Compiled output for production
}
```

ESM modules: The project uses `"type": "module"` in `package.json` . This means all `.ts` files use `import / export` (not `require`). At runtime (`node dist/index.js`), Node.js natively understands ESM without any additional transpilation.

tsx for development: The `npm run chat` command uses `tsx` to run TypeScript directly without pre-compiling, enabling faster iteration during development.

4.8 Docker + Docker Compose — Infrastructure

Services defined in `docker-compose.yml` :

```
services:
  qa-agent:      # The Testa Node.js server
```

```
postgres:      # pgvector/pgvector:pg17
redis:         # redis:8-alpine
```

Key design decisions:

Volume mounts for the automation project:

```
volumes:
  - /Users/uchoudhary/Delivery/magnit/qa/pw-sanity:/app/automation
```

The live Playwright project is bind-mounted into the container. This means the agent can read the real test files and page objects without any sync or copy step. When you edit a spec on the host, the agent immediately sees the change.

Ollama on the host, not in Docker: Running Ollama in Docker on macOS would lose GPU acceleration (Docker on macOS runs Linux in a VM and can't access the Metal GPU). By running Ollama natively on macOS and using `host.docker.internal`, the containers get full GPU performance.

PostgreSQL initialisation:

```
volumes:
  - ./database/init:/docker-entrypoint-initdb.d
```

PostgreSQL's official Docker image automatically runs any `.sql` files in `/docker-entrypoint-initdb.d` on first startup. This creates the `memories` table and HNSW index without any manual setup.

Healthchecks:

```
postgres:
  healthcheck:
    test: ["CMD", "pg_isready", "-U", "qa", "-d", "qa_brain"]
    interval: 5s
    retries: 5
```

The `qa-agent` service uses `depends_on: postgres: condition: service_healthy` so the agent won't start until PostgreSQL is ready to accept connections.

5. Agent Subsystems

5.1 Memory Layer (`memory.ts`)

The memory layer provides the agent's ability to learn and recall information across conversations.

Memory types used by Testa:

- `feature` — product features and behaviours
- `flow` — user journeys and multi-step processes
- `edge-case` — boundary conditions and known bugs
- `architecture` — code structure, page objects, methods
- `fact` — general project facts
- `bug` — known defects and workarounds

- `test-coverage` — what test files exist and what they cover

Write path:

```
User teaches a fact
  ↓
nomic-embed-text converts text → 768-dim vector
  ↓
INSERT INTO memories (content, memory_type, source, confidence, embedding)
```

Read path (semantic search):

```
User asks a question
  ↓
nomic-embed-text converts question → 768-dim vector
  ↓
SELECT ... ORDER BY embedding <=> $queryVector LIMIT 6
  ↓
Return memories sorted by semantic similarity
```

Note on connection management: Currently each `memory.ts` call creates a new `pg.Client`, connects, executes, and disconnects. For a production system this should use a connection pool (`pg.Pool`) to reuse connections.

5.2 Session Layer (`session.ts`)

Manages the rolling conversation window for multi-turn chat.

```
getConversation(sessionId, 10) → last 10 messages from Redis list
addMessage(sessionId, msg)      → RPush + EXPIRE 24h
clearConversation(sessionId)    → DEL the list
```

Sessions are keyed by a UUID generated client-side and stored in `localStorage`. This means each browser tab can have an independent conversation.

5.3 LLM Client (`ollama.ts`)

The thin wrapper around Ollama's REST API.

System prompt architecture:

The system prompt in `chat()` has two layers:

```
Layer 1: Hardcoded persona
  "You are Testa, an AI QA engineer. You specialise in..."

Layer 2: Injected knowledge (dynamic, per-request)
  "CURRENT PRODUCT KNOWLEDGE:
  [feature] Bookings require a BU and Job...
  [architecture] BookingsPage has: startCreateBooking()..."
```

This separation means:

- The persona is stable (hardcoded)

- The knowledge is fresh (fetched from memory each request)
- Adding knowledge doesn't require re-deploying the agent

5.4 Knowledge Ingestion (`ingest.ts`)

Converts large documents into searchable memory chunks.

Chunking strategy:

```
Input text (e.g., a page object file)
↓
Split on blank lines (paragraph boundaries)
↓
If chunk > 400 chars: split on sentence boundaries (?!/.)
↓
Skip chunks < 20 chars (noise)
↓
For each chunk: embed → store in PostgreSQL
```

Why 400 characters? Each chunk should represent one complete idea. Too small: no context. Too large: the embedding averages too many topics, reducing retrieval precision. 400 chars (~80 words) is roughly one paragraph — a natural semantic unit.

The `scripts/ingest-project.mjs` bootstrap script does a one-time deep ingestion of the entire automation project: specs, page objects, types, API services, and hardcoded domain knowledge. After this runs, the agent has a rich knowledge base without anyone having to manually teach it.

5.5 Test Engineering Tools (`testtools.ts`)

Three AI-powered capabilities built on top of the LLM client:

`generateTest(feature, knowledge, options)`

The generation prompt contains nine sections:

1. Role definition (QA engineer for this project)
2. Framework rules (Playwright, TypeScript page objects, `test.step`)
3. Available imports (exact paths, no guessing)
4. Fixture signatures (what `{ page, homepage, ... }` provides)
5. The correct `beforeEach` template for the chosen portal
6. A complete real test as a few-shot example
7. Product knowledge from memory
8. Page object context (relevant methods for this feature)
9. The request (generate a test for `feature`)

The prompt instructs the model to return only TypeScript, no markdown, enabling direct write to disk.

`reviewTest(code, knowledge)`

Reviews against five dimensions: coverage gaps, assertion quality, selector resilience, structural conformance, and the top 3 improvements prioritised by impact.

`analyzeTestResults(output, testCode, knowledge)`

Processes Playwright terminal output and classifies each failure:

Category	Meaning
FLAKY SELECTOR	A CSS/ARIA selector broke — likely a UI change
TIMING ISSUE	waitFor timeout or race condition
DATA SETUP	Test data wasn't created or is in wrong state
PRODUCT BUG	Application behaviour is wrong, not the test
TEST LOGIC	The test assertion or flow is incorrect
ENVIRONMENT	Network, auth, or infrastructure problem

5.6 Automation Browser (automation.ts)

Provides the agent with live, read-only access to the Playwright project.

Security design: `readTestFile()` validates that the resolved path stays within the `automationPath` :

```
const resolved = resolve(automationPath, relativePath);
if (!resolved.startsWith(automationPath)) {
  return null; // path traversal blocked
}
```

Page object relevance scoring for generation context:

```
filename match    = 10 points per keyword
first 50 lines    = 2 points per keyword hit
core pages always = 100 points (login, home, base)
```

This ensures the LLM always gets login/navigation context plus the most relevant feature-specific page object.

6. Prompt Engineering in Testa

The system prompt as a contract

The system prompt is effectively a contract with the LLM. It should:

1. State the role clearly ("You are X who does Y")
2. List hard rules (what not to do)
3. Define output format (especially for code generation)
4. Set the tone and uncertainty handling

Testa's system prompt:

You are Testa, an AI-powered QA engineer specialising in Playwright TypeScript test automation. You help create, review, and debug end-to-end tests.

Rules:

- NEVER invent product behaviour you haven't been told about
- When uncertain: say "I don't know for certain, but based on what I know..."

- Use TypeScript page object pattern
- Tests should follow the existing project structure

Few-shot prompting

For code generation, Testa includes a complete real test as an example. This is the single highest-impact prompt technique for code generation:

Here is an example of a well-structured test in this project:

```
=== EXAMPLE TEST ===
import { expect } from '@playwright/test';
import { test } from '@fixtures/test.fixture';
// ... complete real test ...
=== END EXAMPLE ===
```

Now generate a test for: {feature}

The model learns the exact import style, indentation, fixture pattern, `test.step` usage, and tag format from the example — producing output that is immediately compatible with the project.

Knowledge injection format

The format of injected memories matters:

```
[memory_type] content
```

The `[memory_type]` prefix tells the model the reliability and category of each piece of knowledge.

`[architecture]` facts (code structure) are reliable. `[edge-case]` facts describe unusual scenarios. The model can weight these differently in its reasoning.

7. Weighting, Confidence, and Tuning

Memory confidence scores

Each memory has a `confidence` field (`NUMERIC(4,3)` , range 0.000–1.000).

```
confidence = 1.000 -- verified fact, ingested programmatically
confidence = 0.800 -- user-taught fact, believed but not verified
confidence = 0.500 -- inferred or uncertain
```

Currently the retrieval query doesn't filter by confidence, but a production enhancement would be:

```
-- Only use high-confidence memories
WHERE confidence >= 0.7
ORDER BY embedding <=> $1::vector LIMIT 6

-- Or boost by confidence × similarity
ORDER BY (1 - (embedding <=> $1::vector)) * confidence DESC
```

Similarity threshold tuning

The `recallRelevant` function returns the top K memories regardless of their similarity score. A memory with similarity 0.3 (weakly related) is returned alongside one at 0.9 (strongly related).

A production improvement is to add a minimum similarity threshold:

```
const MIN_SIMILARITY = 0.5;

// After retrieval:
const relevantMemories = memories.filter(m => (m.similarity ?? 0) >= MIN_SIMILARITY);
```

If retrieval is returning irrelevant memories, increase this threshold. If the agent is answering "I don't know" for things it should know, decrease it.

K (number of retrieved memories) tuning

Currently `recallRelevant(message, 6)` — always fetches 6 memories.

- Too few (K=2): agent misses relevant knowledge, gives incomplete answers
- Too many (K=20): relevant memories are diluted by weakly-related ones, can confuse the model, and consumes more of the context window

The right K depends on the domain size and query specificity. 6 is a reasonable default for a medium-sized knowledge base (~500 memories). For a very large knowledge base, consider two-stage retrieval: fetch 20, re-rank, take the top 6.

Temperature by task

Different tasks benefit from different temperatures. Since Ollama accepts `options` in the chat request, you can tune per endpoint:

```
// For deterministic code generation – low temperature
body: JSON.stringify({
  model: chatModel,
  messages: ...,
  options: { temperature: 0.2 }
})

// For brainstorming test cases – higher temperature
options: { temperature: 0.7 }

// For factual Q&A – medium temperature
options: { temperature: 0.4 }
```

Model selection tradeoffs

Model	Size	Speed (M3 Pro)	Code quality	Context
llama3.2:3b	2GB	~60 tok/s	Good	128K
llama3.2:8b	5GB	~25 tok/s	Better	128K

llama3.1:70b	40GB	~3 tok/s	Excellent	128K
codellama:13b	8GB	~15 tok/s	Best for code	16K
qwen2.5-coder:7b	5GB	~25 tok/s	Excellent for code	32K
mistral:7b	4GB	~30 tok/s	Good	32K

For Testa's use case, `qwen2.5-coder:7b` is worth evaluating as a code-focused alternative to `llama3.2` — it's specifically fine-tuned on code generation tasks.

8. Technology Alternatives — Pros and Cons

8.1 LLM Runtime Alternatives

Option A: Ollama (current)

Pros	Free. Privacy. Simple API. Apple Silicon GPU. Model management built-in.
Cons	Mac/Linux only. No Windows GPU support. Limited monitoring. No rate limiting.

Option B: LM Studio

Pros	GUI for model management. OpenAI-compatible API. Windows/Mac/Linux. Good for less technical users.
Cons	Closed source. GUI-first (harder to script). Slightly worse performance than Ollama on Apple Silicon.

Option C: llama.cpp directly

Pros	Maximum performance and control. No abstraction layer. Can be embedded as a C library.
Cons	No model management. No API server (need to add). Complex setup. Not beginner-friendly.

Option D: OpenAI / Anthropic API

Pros	Best model quality (GPT-4, Claude Sonnet). No hardware requirements. Managed scaling.
Cons	Cost (\$0.01–\$0.06 per 1K tokens). Data leaves your network. Vendor dependency. Requires internet.

Option E: Hugging Face Inference API

Pros	Huge model selection. Free tier. Hosted option.
Cons	Rate limits on free tier. Cold start latency. Data leaves network.

8.2 Vector Database Alternatives

Option A: PostgreSQL + pgvector (current)

Pros	No additional service. Combine vector + relational queries. ACID. Great tooling. HNSW built-in.
Cons	Not as fast as dedicated vector DBs at massive scale (10M+ vectors). Requires pgvector extension.

Option B: Chroma

Pros	Purpose-built for embeddings. Simple Python/JS API. Easy to embed in an app. Free open source.
Cons	Another service to run. No relational queries. Less mature than PostgreSQL.
Best for:	Python-first AI applications, prototyping

Option C: Qdrant

Pros	Extremely fast. Built in Rust. Advanced filtering. Payload storage. REST + gRPC. Self-hosted or cloud.
Cons	Another service. More complex config. Overkill for small knowledge bases.
Best for:	Production vector search at scale (100K+ vectors)

Option D: Weaviate

Pros	Built-in embedding models. GraphQL query API. Multi-modal (text + images).
Cons	Complex setup. Heavy resource usage. GraphQL learning curve.
Best for:	Multi-modal AI applications

Option E: SQLite + sqlite-vec

Pros	Zero server setup. Single file. Embedded in the app. Great for small knowledge bases.
Cons	No concurrent writes. Limited scaling. Less mature vector support.
Best for:	Single-user agents, development, laptops without Docker

8.3 Session Store Alternatives

Option A: Redis (current)

Pros	Microsecond reads/writes. Built-in TTL. List data structure is perfect for message history. Mature.

Cons	Another service to run. In-memory (data lost on restart without persistence).
-------------	---

Option B: In-memory Map (e.g., `Map<string, Message[]>`)

Pros	Zero infrastructure. Simplest possible implementation.
Cons	Lost on server restart. Doesn't work with multiple agent instances. No TTL without a timer.
Best for:	Single-user prototypes, development

Option C: PostgreSQL sessions table

Pros	Persistent across restarts. Already have PostgreSQL. Queryable.
Cons	Slower than Redis. More complex schema. Overkill for ephemeral conversation data.

Option D: Upstash Redis (managed)

Pros	No server to run. HTTP API (works anywhere). Free tier.
Cons	Data leaves network. Cost at scale.

8.4 Agent Server Alternatives

Option A: Node.js raw http (current)

Pros	No dependencies. Tiny. Complete control. Fast startup.
Cons	Manual routing. No middleware. Error handling from scratch.

Option B: Hono (TypeScript-first web framework)

Pros	Tiny (~14KB). Type-safe routing. Runs on Node, Bun, Cloudflare Workers, Deno. OpenAPI built-in.
Cons	Another dependency. (Though a very small one.)
Best for:	Production agents where routing grows complex

Option C: Fastify

Pros	Fastest Node.js HTTP framework. Built-in validation (JSON Schema). Great plugin ecosystem.
Cons	More complex than raw http. Plugin architecture has a learning curve.
Best for:	High-throughput agents with complex middleware needs

Option D: Python FastAPI

Pros	First-class ML ecosystem (LangChain, LlamaIndex, transformers). Async native. Auto OpenAPI docs.
Cons	Requires Python runtime. Less natural for developers who know TypeScript.
Best for:	Agents with heavy ML preprocessing (fine-tuning, custom embeddings, audio/vision)

Option E: LangChain / LlamaIndex

Pros	Pre-built RAG pipelines, agent tools, memory systems. Large community. Many integrations.
Cons	Heavy abstraction — debugging is hard. Rapid version changes. Can be over-engineered for simple agents.
Best for:	Complex multi-step agents with many tools; teams that want a framework

8.5 Embedding Model Alternatives

Model	Dimensions	Size	Quality	Speed
nomic-embed-text (current)	768	274MB	Good	Fast
all-MiniLM-L6-v2	384	80MB	OK	Very fast
mxbai-embed-large	1024	670MB	Very good	Medium
text-embedding-3-large (OpenAI)	3072	Cloud	Excellent	Fast (API)
jina-embeddings-v3	1024	570MB	Excellent	Medium

For upgrading: if retrieval quality needs improvement, try `mxbai-embed-large` (1024-dim). Note: changing the embedding model requires re-embedding all existing memories (delete and re-ingest) because the vector spaces are incompatible.

9. Building a New Custom Agent — Blueprint

Here is a step-by-step guide to building a new domain-specific agent using the same architecture as Testa.

Step 1: Define the domain and capabilities

Answer these questions before writing code:

- What domain does the agent specialise in? (QA, DevOps, finance, legal, etc.)
- What are the 3–5 capabilities it needs? (Testa: generate, review, analyze, teach)
- What project/documents will it need to read?
- Who will use it and how? (Web UI, Slack bot, CLI, API)

Step 2: Stand up infrastructure

Copy the `docker-compose.yml` from this project. You need only three services:

```
pgvector/pgvector:pg17 → long-term memory
redis:8-alpine          → conversation sessions
```

And your agent service itself. Ollama runs on the host.

Run the database init SQL to create the memories table and HNSW index.

Step 3: Define memory types

Design memory types for your domain. Examples:

```
// QA agent (current):  feature, flow, architecture, bug, edge-case
// DevOps agent:        incident, runbook, service, dependency, alert
// Legal agent:          regulation, case, clause, jurisdiction, precedent
// Finance agent:        rule, product, rate, limit, exception
```

Step 4: Build the LLM client

The Ollama client from `ollama.ts` is reusable as-is. Only change:

- The system prompt persona
- The model name (via env var)

```
const systemPrompt = `You are [Name], an AI [role] specialising in [domain].

Rules:
1. NEVER invent [facts/behaviour/regulations] you haven't been told about
2. When uncertain, say "I'm not certain, but..."
3. [Domain-specific rules]`;
```

Step 5: Implement the memory module

Copy `memory.ts` exactly. It is domain-agnostic. The only thing you might customise:

- Embedding model (if switching from nomic-embed-text)
- Add a `confidence` filter in `recallRelevant()` for high-trust-only retrieval
- Add a `similarity_threshold` filter to exclude weakly-related memories

Step 6: Design the domain-specific tools

Each domain-specific capability is a function in a `tools.ts` equivalent:

```
export async function generateRunbook(
  incident: string,
  knowledge: string
): Promise<string> {
  const prompt = `
  You are a SRE. Generate a Markdown runbook for this incident:
```

```

    ${incident}

    === KNOWN CONTEXT ===
    ${knowledge}

    Include: summary, diagnosis steps, remediation steps, rollback procedure.
    `;
    return chat([ { role: "user", content: prompt } ], "");
  }

```

Step 7: Build the HTTP server

Copy `index.ts` . Replace domain-specific handlers:

- Keep: `/chat` , `/memory` , `/ingest/text` , `/health` , `/session/clear`
- Replace: `/test/generate` with your domain-specific endpoints
- Add any read-only data access endpoints your domain needs

Step 8: Ingest knowledge

Write an ingestion script equivalent to `scripts/ingest-project.mjs` . This should:

1. Crawl your domain's documents/data sources
2. Extract structured text (remove boilerplate HTML, navigation, etc.)
3. POST to `/ingest/text` to chunk and embed

The better your ingestion, the better the agent's answers. Invest time here.

Step 9: Build or adapt the UI

The `public/index.html` is self-contained. Customise:

- Brand name and colours (CSS custom properties)
- Sidebar tool sections (replace test generation with your tools)
- Memory type options in the "Teach" dropdown

Step 10: Test and tune

1. Ask the agent questions it should know from ingested knowledge
2. Check which memories are being retrieved (`memoriesUsed` in the response)
3. If quality is poor:
 - Improve chunking (smaller chunks = more precise retrieval)
 - Add more context to each chunk (include file/section headers)
 - Tune `K` (number of recalled memories)
 - Add a similarity threshold
 - Switch to a better embedding model
4. Check the system prompt — unclear rules produce unpredictable behaviour

10. Glossary

Term	Definition
Attention	The mechanism in Transformers that allows each token to "attend to" (reference) all other tokens in the context. The foundation of LLM

	intelligence.
Backpropagation	The algorithm used during training to propagate prediction errors backward through the network and update weights.
Chunking	Splitting large documents into smaller segments for embedding and retrieval.
Cosine similarity	A measure of the angle between two vectors, used to compare semantic meaning. Ranges from -1 (opposite) to 1 (identical).
Embedding	A fixed-size vector representation of a piece of text that captures its semantic meaning.
GGUF	The file format used by llama.cpp and Ollama for quantised LLM weights.
HNSW	Hierarchical Navigable Small World. A graph-based approximate nearest-neighbour index.
INT4/INT8 quantisation	Compressing model weights from 32-bit floats to 4-bit or 8-bit integers to reduce memory and increase speed.
K (in top-K retrieval)	The number of most similar memories to retrieve for a given query.
LLM	Large Language Model. A neural network trained on massive text datasets to predict and generate language.
Parameters / weights	The learned numeric values inside a neural network. A 3B model has 3 billion of them.
pgvector	PostgreSQL extension that adds a vector column type and approximate nearest-neighbour search operators.
Prompt	The input given to an LLM. Consists of a system message (persona/rules) and user messages.
Quantisation	Reducing the numeric precision of model weights (float32 → int4) to reduce RAM and increase speed.
RAG (Retrieval-Augmented Generation)	Technique of retrieving relevant documents/memories and injecting them into the prompt before generating an answer.
Semantic search	Searching by meaning (via embeddings) rather than by keyword matching.
Temperature	Controls randomness in token sampling. 0 = deterministic, 1+ = creative/random.
Token	The unit of text an LLM processes. Roughly 0.75 words on average. "Hello world" = 2 tokens.
Vector	An ordered list of numbers. In AI, used to represent the meaning of text in a mathematical space.

Vector database

A database optimised for storing and searching high-dimensional vectors using approximate nearest-neighbour algorithms.